Assignment 4

Aaron Bussche David Magaram Jason Kasdiran
Group 28

April 15, 2026

Introduction

Evolutionary algorithms (EA) are a population-based optimization method. They are based on natural selection and work by exploring new solution areas and iteratively refining good already found solutions, where candidate solutions are improved by mutating and crossover. A common issue is the trade-off between exploration and exploitation. Too much exploration can lead to slow convergence, while too much exploitation can cause suboptimal solutions. Despite this, EAs are still very useful, especially for larger, more complex problems. However, their performance depends on parameter configurations such as mutation rate, population size, and selection pressure.

By investigating the traveling salesman problem and string search, this project explores how different parameter configurations influence convergence speed, population diversity, and algorithm performance. It also aims to explore how mutation rate and selection pressure shape the exploration-exploitation trade-off in these settings, and whether a 2-opt local search can outperform a standard EA algorithm on the traveling salesman problem under fair settings.

Methods

This project will implement a genetic algorithm (GA) for string search and memetic algorithm (MA) for traveling salesman problem. This section describes the experimental setup and design choices. All algorithms were run single-threaded on a desktop system with a AMD Ryzen 5800x processor.

String search

A string search genetic algorithm was implemented and the used parameters are displayed in Table 1. "NaturalComputing" was used as the target string with a length L = 16, containing both lowercase and uppercase letters from an alphabet $|\Sigma|$. Since fitness is the number of characters that match the target, the maximum fitness is 16.

The algorithm aims to match the target string by evolving a population of N = 200 individuals. Each generation follows a few simple steps. First, during the tournament selection phase, a random subset K is drawn from the population N and the fittest string is kept for reproduction.

Larger K values mean more potentially high-fitness strings, which can lead to the selection of only the best performers, working as a filter against weaker candidates. While it usually leads to faster convergence, it can also reduce diversity as the entire population will mimic the strongest strings.

Then, each pair undergoes single-point crossover with a probability $p_c = 1$, producing 2 new offsprings. Crossover selects a random index and swaps the parent's characters beyond that cutoff. Additionally, every character of the offspring has a small mutation rate μ to randomly become one of the letters from the alphabet $|\Sigma|$. Finally, the entire parent generation is replaced with the new offspring generation.

Population diversity was measured using the mean pairwise Hamming distance computed on a random subsample of the population and the average position-wise Shannon entropy

$$H_i = -\sum_j f_{ij} \log f_{ij},$$

where f_{ij} is the frequency of character j at position i. Furthermore, run is terminated if it reaches target string or the maximum generations $G_{max} = 200$. In case the target is not reached, run is recorded as $t_{finish} = G_{max}$.

Parameter	**Value**
String length L	16
Alphabet $\|\Sigma\|$	52
Population size N	200
Tournament selection K	$\{2, 5\}$
Crossover probability p_c	1
Mutation rate μ	$\{0,\ 1/L,\ 3/L\}$ + sweep from 0/L to 10/L
Runs	10
Max generations $G_{\max}$	200

Table 1: Parameters for Exercise B1.

Traveling Salesman Problem

For the TSP, both a simple evolutionary algorithm (EA) and a memetic algorithm (MA) were implemented. Each solution is encoded as a permutation of city indices representing a tour. The simple EA uses ordered crossover (OX): a random contiguous segment is copied from one parent and the remaining cities are filled in the order they appear in the second parent. Mutation is applied via swap mutation, where two randomly selected positions in the tour are exchanged. Tournament selection ($K = 5$) is used to select parents, and generational replacement with no elitism is applied.

The memetic algorithm extends the EA by applying 2-opt local search to each individual after mutation. The 2-opt procedure iteratively reverses sub-sequences of the tour to eliminate crossing edges, repeating until no improving swap can be found. This Lamarckian approach allows individuals to locally optimize their fitness before evaluation, which typically accelerates convergence. Both algorithms were evaluated on the TSPLIB benchmark instance *eil51* and on the 50-city dataset from the Symmetric Traveling Salesman Problem benchmark instances available at http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/.

Table 3 displays the used parameters. These parameters were chosen based on the previous exercise and further adjusted based on testing. The crossover probability was lowered to 0.8. This is high enough to help produce new better solutions, but not too high as to break already good solutions. Moreover, mutation was set to 0.3 to explore the area around solutions, and tournament selection was set to 5 as it accelerates convergence and works fairly well. Overall, these parameter settings provide balance between exploration and exploitation.

Parameter	**Value**
Population size N	100
Tournament selection K	5
Crossover probability p_c	0.8
Mutation rate μ	0.3
Local search	2-opt
Runs	10
Max generations $G_{\max}$	200
Datasets	eil51 (TSPLIB), 50-city instance
Stopping criterion	Fixed generations / wall-clock budget
2-opt termination	Until no improving 2-swap remains

Table 2: Parameters for Exercise B2.

Results

B1

In this section, we aim to answer whether and how the mutation rate μ trades convergence speed against diversity, and how the tournament size K impacts this. We first run the GA 10 times with the baseline values given by the exercise ($K = 2$, $\mu = 1/L$, $N = 200$) and then experiment with varying the values of μ and later K.

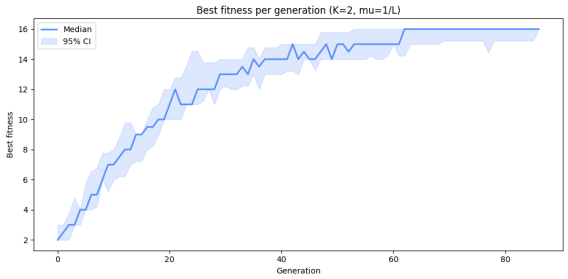

Figure 1: Results of running the GA 10 times with baseline values ($K = 2$, $\mu = 1/L$, $N = 200$). Blue line: median over 10 samples; shaded area: confidence interval.

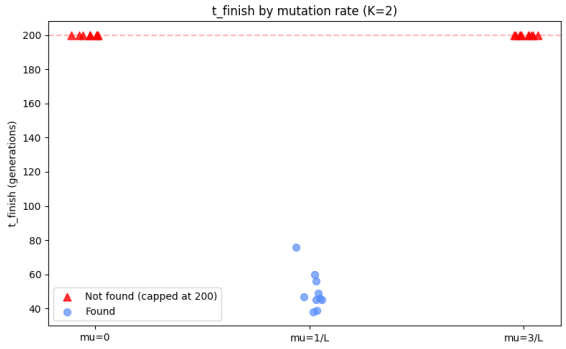

Figure 2: Three different mutation rates with $K = 2$. Only $\mu = 1/L$ finds the target before the 200-generation cutoff.

With the baseline values, the algorithm finds correct results within 86 generations for all 10 samples, but usually earlier (Figure 1). The average generation count till t_{finish} is 61.9. When changing μ to 0 or $3/L$, but keeping the other parameters equal, solutions are not found within 200 generations (Figure 2).

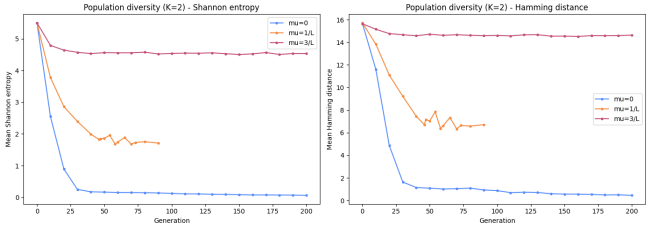

Figure 3: Measuring the diversity of runs in two different ways, with mean position-wise Shannon entropy and pairwise Hamming distance between strings over 10 runs.

$\mu = 0$	$\mu = 3/L$
`BaturalCrmputing` [14]	`NatusYOiHxpgJijg` [7]
`BaturalCrmputing` [14]	`UttKhaaYojputioh` [7]
`BaturalCSmputing` [14]	`iattIYvCojpgOTIg` [6]
`BaturalCrmputing` [14]	`iattIBYCojpgVbnk` [6]

Table 3: Top 4 individuals at generation 100 ($K = 2$, fitness in brackets). The $\mu = 1/L$ case is omitted because the population has already converged on the target.

Measuring the diversity of runs with different values of μ, both Shannon entropy and Hamming distance show that $\mu = 0$ leads to low diversity, $\mu = 3/L$ leads to very high diversity, while $\mu = 1/L$ manages to strike a middle ground (and reach the target) (Figure 3). In the qualitative results, we can observe that $\mu = 0$ collapses to near-identical copies stuck at a local optimum; $\mu = 3/L$ stays far from the target `NaturalComputing` with very different strings (Table 3).

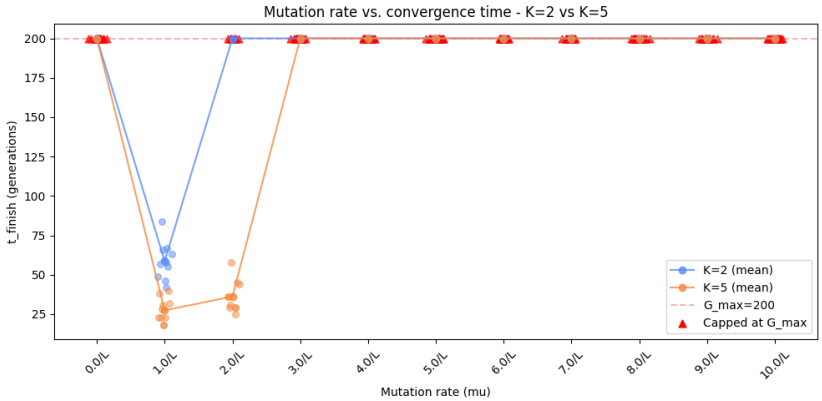

Figure 4: Joint graph exploring different μ for both $K = 2$ and $K = 5$. Runs are capped at 200 generations.

We repeat the μ sweep of Figure 2, varying K and μ more to see how a higher K interacts with mutation rate. $K = 5$ leads to results faster and can also converge with $\mu = 2/L$, though $\mu = 1/L$ remains optimal. As higher tournament size increases selection pressure, it can deal with more noise. Other μ's than $1/L$ for $K = 2$ and $1/L$ and $2/L$ for $K = 5$ do not lead to convergences.

B2

In this section, we aim to explore whether 2-opt local search improves TSP performance beyond the extra compute it costs. We test on both the given TSP problem and the TSPLIB eli51 dataset. We first test both the EA and the MA implementations with an identical amount of generations.

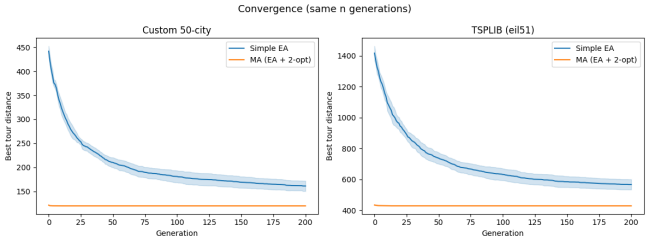

Figure 5: Distance vs. generations, with the mean in blue/orange and the standard deviation in light blue/orange over 10 runs.

	EA		MA	
Dataset	best	std	best	std
eil51	519.13	33.09	428.87	0.25
given	150.76	10.69	119.94	0.00

Table 4: Best final tour length and standard deviation over 10 runs, within the 200-generation limit.

On both datasets the MA reaches shorter distances within the 200-generation limit (Figure 5, Table 4), and its low final standard deviation shows it consistently converges to the same solution across runs. On the other hand, the EA's spread indicates it still gets stuck in different local optima at the cut-off.

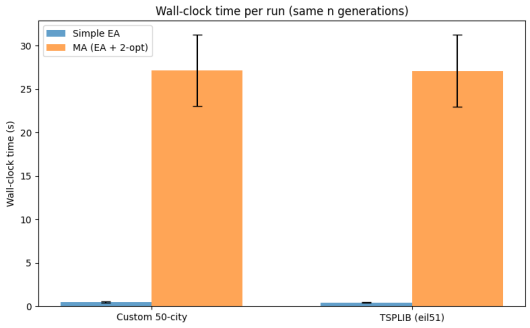

Figure 6: Time comparison between EA and MA both running for 200 generations, mean score over 10 runs.

There is an important caveat: Each MA generation includes a full 2-opt local search over every individual in the population, making it far more computationally expensive than a single EA generation. As generations are not equivalent in computational power needed, it is therefore not fair to compare the algorithms by number of generations. We considered comparing them on number of operations, but 2-opt search of MA does cheap evaluations within a solution's neighborhood, so comparing them directly with full-tour evaluations of EA would again be unfair. We therefore decided to compare wall-clock time that, while influenceable by all kinds of factors and therefore not overly scientific, provides a good baseline for real-use performance, especially if averaged over 10 runs each.

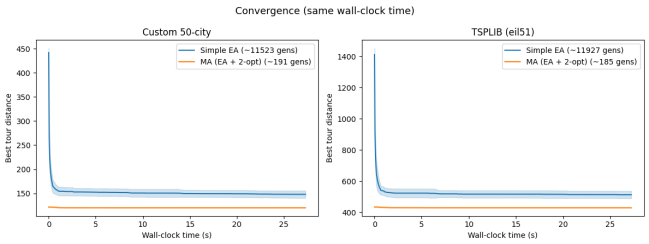

Figure 7: Distance vs. wall-clock time, mean over 10 runs for both datasets.

	EA		MA	
Dataset	best	std	best	std
eil51	482.05	22.99	428.87	0.05
given	136.89	7.57	119.94	0.00

Table 5: Best final tour length and std over 10 runs, same wall-clock budget. In that budget the EA completes ∼11500-12000 generations while the MA reaches only ∼185-190.

Even under a fair wall-clock comparison, the MA consistently reaches shorter and more consistent tour distances than the simple EA on both datasets (Table 5). While the gap has considerably shrunk from comparing by number of generations, EA's curve also flattens to near its best result in a comparable time as MA (Figure 7). The addition of local search therefore improves performance beyond what the extra computation alone can explain. 2-opt allows individuals to escape local optima that trap the plain EA efficiently, reducing both mean tour length and variance across runs.

Discussion

Several limitations of this study should be noted. All experiments were run on a single machine, and the results may differ on other hardware or with a more optimised 2-opt implementation, which are all factors which may affect the wall-clock time. With only 10 runs per condition the confidence intervals are modest, particularly at boundary mutation rates where success is inconsistent, and formal statistical tests (e.g. Wilcoxon rank-sum) would be needed to make stronger claims. Only two TSP instances were tested and only one target string was used for the string search task, so the generalisability of the findings is limited. Regarding threats to validity, 2-opt is a heuristic specifically tailored to tour optimisation problems, so the advantage of the MA over the plain EA may not transfer to other combinatorial problems where no efficient local search is available. Additionally, censoring t_{finish} at $G_{\max} = 200$ when the target is not found biases mean convergence time downward, meaning the true performance gap between mutation rates is likely larger than reported. Future work could test the MA on larger TSPLIB instances to assess scalability, vary the intensity of local search (e.g. full 2-opt vs. a fixed number of improvement steps, or Baldwinian rather than Lamarckian learning), and explore adaptive mutation rates that adjust μ dynamically as population diversity changes.

Conclusion

This project shows that small changes in EA design choices have large and predictable effects on performance. For string search, the mutation rate $\mu = 1/L$ is optimal as it provides enough exploration

to maintain diversity and avoid premature convergence, without being so disruptive that accumulated progress is lost. Increasing tournament size to $K = 5$ raises selection pressure, accelerating convergence and shifting the viable mutation range upward, though the optimum remains $\mu = 1/L$. For the TSP, the memetic algorithm consistently outperforms the plain EA in both solution quality and consistency, and this advantage holds even under a fair wall-clock comparison that accounts for the roughly 25 times higher cost per MA generation. Furthermore, 2-opt local search allows individuals to escape the local optima that cause high variability in the plain EA. The take-home message is that local search is worth its computational cost for TSP, while for string search there is a narrow optimal mutation rate whose precise value shifts with selection pressure. Both findings reflect the same underlying principle that effective evolutionary search requires a carefully tuned balance between exploration and exploitation.

Appendix

Part A

A1

What can you conclude about the effects of fitness scaling on selection pressure?

Adding constants raises all fitness all values by that amount. So it smooths out all the values. For example, when we compare f2 (which only squares X) and f4 (which squares X and adds 20), we can see that the probabilities of f4 are closer together. Probabilities are increased and reduced, so there is a much smaller gap between them, which also reduces selection pressure. Multiplying by a constant does not change the selection probabilities. If we compare f2 and f3, we can see that the probabilities stay the same even if we scale it. So scaling fitness does not have an effect on the selection pressure.

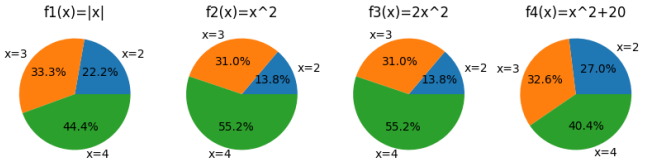

Figure 8: Selection probability of the three individuals for each function.

x	f1	p1	f2	p2	f3	p3	f4	p4
2	2	0.2222	4	0.1379	8	0.1379	24	0.2697
3	3	0.3333	9	0.3103	18	0.3103	29	0.3258
4	4	0.4444	16	0.5517	32	0.5517	36	0.4045

Table 6: Fitness function value and selection probability for each individual and each function.

A2

Does the algorithm find the optimum? Yes. It is able to reach fitness 100, meaning all 100 bits were successfully set to 1.

What do you see? The version without selection is stuck at around 50 and never converges. The version with selection keeps steadily climbing, ultimately reaching 100.

Can you conclude anything on the difference between these two versions based on the above results? If yes, why? If not, what should you do to make a fair comparison? No. We would need multiple runs as EAs are stochastic and each run produces different results because of initialization and random mutation. To make it fair, we should run each version multiple times and use averages.

Is there a difference in performance? Yes. It is apparent that there is a difference in performance between the versions. Across all the runs, the version with selection always converges to fitness 100 while the version without selection is always near 50 through all the 1500 generations. The algorithm has no memory of good solutions and cannot improve.

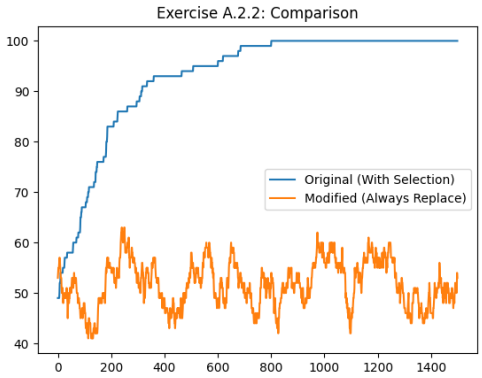

Figure 9: Fitness against generations.

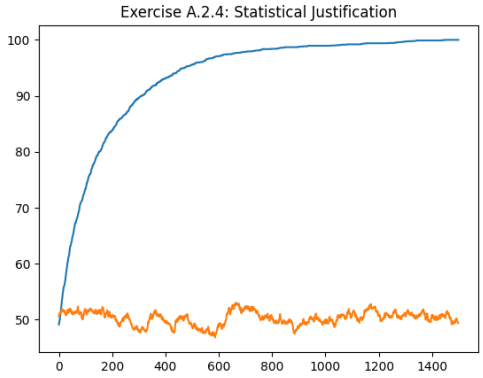

Figure 10: Averages of fitness vs generations across multiple runs.

A Code

All experiments were conducted in Python (version 3.12) and Jupyter notebooks.

The .zip file includes the following files: `experiments_b1.py`, `experiments_b2.py`, `plotting_b1.py`, `plotting_b2.py`, `partA.ipynb`, `partB1.ipynb`, `partB2.ipynb`, `eil51.tsp`, and `file-tsp.txt`.

To reproduce the results, all scrips and files should be located in the same working directory as the notebooks, which make use of them to execute experiments and generate results.